1. Problem Statement – Student Management System

Create a **database** named StudentDB to manage student, department, course, and enrollment information such as,

Department(dept_id, DeptName, HOD_Name)

Student(RegID, rollno, fname, mname, lname, gender, dept_id)

Course(CID, course_name, credit)

Enrolment(enid, RegID, DeptID)

During table creation add constraint such as Primary Key, Foreign Key, Unique, Not Null, Check, and Default constraints

1) Add a new column email to the **Student** table
2) Modify the column size of course_name in **Course** table to VARCHAR(120)
3) Rename table **Enrollment** to **StudentEnrollment**
4) Rename column CID in **course** table to course_no
5) Create an index on the dept_id column of the **Student** table

ANS =
```
mysql> CREATE DATABASE StudentDB;
Query OK, 1 row affected (0.08 sec)

mysql> USE StudentDB;
Database changed
mysql> CREATE TABLE Department (
    ->     dept_id INT PRIMARY KEY,
    ->     DeptName VARCHAR(100) NOT NULL UNIQUE,
    ->     HOD_Name VARCHAR(100) NOT NULL
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE Student (
    ->     RegID INT PRIMARY KEY,
    ->     rollno INT UNIQUE NOT NULL,
    ->     fname VARCHAR(50) NOT NULL,
    ->     mname VARCHAR(50),
    ->     lname VARCHAR(50) NOT NULL,
    ->     gender CHAR(1) CHECK (gender IN ('M', 'F')),
    ->     dept_id INT,
    ->     FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Course (
    ->     CID INT PRIMARY KEY,
    ->     course_name VARCHAR(80) NOT NULL,
    ->     credit INT DEFAULT 3 CHECK (credit > 0)
    -> );
```

```
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Enrollment (
    ->     enid INT PRIMARY KEY,
    ->     RegID INT NOT NULL,
    ->     DeptID INT NOT NULL,
    ->     FOREIGN KEY (RegID) REFERENCES Student(RegID),
    ->     FOREIGN KEY (DeptID) REFERENCES Department(dept_id)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE Student
    -> ADD email VARCHAR(100);
Query OK, 0 rows affected (0.15 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE Course
    -> MODIFY course_name VARCHAR(120);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> RENAME TABLE Enrollment TO StudentEnrollment;
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE Course
    -> CHANGE COLUMN CID course_no INT;
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_dept_id ON Student(dept_id);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SHOW TABLES;
+--------------------+
| Tables_in_studentdb |
+--------------------+
| course             |
| department         |
| student            |
| studentenrollment  |
```

```
+--------------------+
4 rows in set (0.03 sec)

mysql> DESC Student;
+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| RegID   | int         | NO   | PRI | NULL    |       |
| rollno  | int         | NO   | UNI | NULL    |       |
| fname   | varchar(50) | NO   |     | NULL    |       |
| mname   | varchar(50) | YES  |     | NULL    |       |
| lname   | varchar(50) | NO   |     | NULL    |       |
| gender  | char(1)     | YES  |     | NULL    |       |
| dept_id | int         | YES  | MUL | NULL    |       |
| email   | varchar(100)| YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
8 rows in set (0.01 sec)

mysql> DESC Course;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| course_no   | int         | NO   | PRI | NULL    |       |
| course_name | varchar(120)| YES  |     | NULL    |       |
| credit      | int         | YES  |     | 3       |       |
+-------------+-------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> DESC StudentEnrollment;
+--------+------+------+-----+---------+-------+
| Field  | Type | Null | Key | Default | Extra |
+--------+------+------+-----+---------+-------+
| enid   | int  | NO   | PRI | NULL    |       |
| RegID  | int  | NO   | MUL | NULL    |       |
| DeptID | int  | NO   | MUL | NULL    |       |
+--------+------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql>
```

2. Problem Statement – Bank Database
   Create a database named **BankDB** to manage bank, branch, customer, and account information such as:
   Bank (bank_id,bank_name,ho_address,conttactno)
   Branch(branch_id,bank_id,branch_name,ifsc,city,state,picode)
   Customer(customerid,fname,mname,lname,gender,dob,phone,create_on)
   Account(account_id,customer_id,branch_id,account_no,account_type,balance,status, open_on)

During table creation, apply **Primary Key, Foreign Key, Unique, Not Null, Check, and Default** constraints as appropriate.

1. Add a new column email (VARCHAR(120), UNIQUE) to the Customer table.
2. Modify the column size of branch_name in the Branch table to VARCHAR(120).
3. Rename table Account to CustomerAccount.
4. Rename column ifsc in the Branch table to ifsc_code.
5. Create an index on the branch_id column of the CustomerAccount (formerly Account) table to speed up lookups by branch.

ANS= create database BankDB;

Query OK, 1 row affected (0.01 sec)

mysql> use BankDB;

Database changed

```
mysql> CREATE TABLE Bank (
    ->     bank_id INT PRIMARY KEY,
    ->     bank_name VARCHAR(100) NOT NULL UNIQUE,
    ->     ho_address VARCHAR(200) NOT NULL,
    ->     contactno VARCHAR(15) NOT NULL CHECK (contactno REGEXP '^[0-9]{10,15}$')
    -> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> CREATE TABLE Branch (
    ->     branch_id INT PRIMARY KEY,
```

```
    ->     bank_id INT NOT NULL,
    ->     branch_name VARCHAR(100) NOT NULL,
    ->     ifsc VARCHAR(20) UNIQUE NOT NULL,
    ->     city VARCHAR(100) NOT NULL,
    ->     state VARCHAR(100) NOT NULL,
    ->     pincode VARCHAR(10) CHECK (pincode REGEXP '^[0-9]{6}$'),
    ->     FOREIGN KEY (bank_id) REFERENCES Bank(bank_id)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Customer (
    ->     customerid INT PRIMARY KEY,
    ->     fname VARCHAR(50) NOT NULL,
    ->     mname VARCHAR(50),
    ->     lname VARCHAR(50) NOT NULL,
    ->     gender CHAR(1) CHECK (gender IN ('M','F')),
    ->     dob DATE NOT NULL,
    ->     phone VARCHAR(15) UNIQUE NOT NULL,
    ->     create_on DATE DEFAULT (CURRENT_DATE)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Account (
    ->     account_id INT PRIMARY KEY,
    ->     customer_id INT NOT NULL,
    ->     branch_id INT NOT NULL,
    ->     account_no VARCHAR(20) UNIQUE NOT NULL,
```

```
    ->     account_type VARCHAR(20) CHECK (account_type IN
('SAVINGS','CURRENT','FD','RD')),

    ->     balance DECIMAL(12,2) DEFAULT 0 CHECK (balance >= 0),

    ->     status VARCHAR(10) DEFAULT 'ACTIVE' CHECK (status IN
('ACTIVE','INACTIVE','CLOSED')),

    ->     open_on DATE DEFAULT (CURRENT_DATE),

    ->     FOREIGN KEY (customer_id) REFERENCES Customer(customerid),

    ->     FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)

    -> );

Query OK, 0 rows affected (0.03 sec)


mysql> ALTER TABLE Customer

    -> ADD email VARCHAR(120) UNIQUE;

Query OK, 0 rows affected (0.05 sec)

Records: 0  Duplicates: 0  Warnings: 0


mysql> ALTER TABLE Branch

    -> MODIFY branch_name VARCHAR(120);

Query OK, 0 rows affected (0.06 sec)

Records: 0  Duplicates: 0  Warnings: 0


mysql> RENAME TABLE Account TO CustomerAccount;

Query OK, 0 rows affected (0.02 sec)


mysql> ALTER TABLE Branch

    -> CHANGE COLUMN ifsc ifsc_code VARCHAR(20);

Query OK, 0 rows affected (0.05 sec)

Records: 0  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE INDEX idx_branch_id ON CustomerAccount(branch_id);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SHOW TABLES;
+-----------------+
| Tables_in_bankdb |
+-----------------+
| bank            |
| branch          |
| customer        |
| customeraccount  |
+-----------------+
4 rows in set (0.00 sec)

mysql> DESC Bank;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| bank_id    | int          | NO   | PRI | NULL    |       |
| bank_name  | varchar(100) | NO   | UNI | NULL    |       |
| ho_address | varchar(200) | NO   |     | NULL    |       |
| contactno  | varchar(15)  | NO   |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

```
mysql> DESC Branch;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| branch_id   | int          | NO   | PRI | NULL    |       |
| bank_id     | int          | NO   | MUL | NULL    |       |
| branch_name | varchar(120) | YES  |     | NULL    |       |
| ifsc_code   | varchar(20)  | YES  | UNI | NULL    |       |
| city        | varchar(100) | NO   |     | NULL    |       |
| state       | varchar(100) | NO   |     | NULL    |       |
| pincode     | varchar(10)  | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
7 rows in set (0.00 sec)

mysql> DESC Customer;
+------------+-------------+------+-----+-----------+-------------------+
| Field      | Type        | Null | Key | Default   | Extra             |
+------------+-------------+------+-----+-----------+-------------------+
| customerid | int         | NO   | PRI | NULL      |                   |
| fname      | varchar(50) | NO   |     | NULL      |                   |
| mname      | varchar(50) | YES  |     | NULL      |                   |
| lname      | varchar(50) | NO   |     | NULL      |                   |
| gender     | char(1)     | YES  |     | NULL      |                   |
| dob        | date        | NO   |     | NULL      |                   |
| phone      | varchar(15) | NO   | UNI | NULL      |                   |
| create_on  | date        | YES  |     | curdate() | DEFAULT_GENERATED |
| email      | varchar(120)| YES  | UNI | NULL      |                   |
```

```
+------------+-------------+------+-----+----------+-----------------+
```

9 rows in set (0.00 sec)

```
mysql> DESC CustomerAccount;
+-------------+--------------+------+-----+----------+-----------------+
| Field       | Type         | Null | Key | Default  | Extra           |
+-------------+--------------+------+-----+----------+-----------------+
| account_id  | int          | NO   | PRI | NULL     |                 |
| customer_id | int          | NO   | MUL | NULL     |                 |
| branch_id   | int          | NO   | MUL | NULL     |                 |
| account_no  | varchar(20)  | NO   | UNI | NULL     |                 |
| account_type| varchar(20)  | YES  |     | NULL     |                 |
| balance     | decimal(12,2)| YES  |     | 0.00     |                 |
| status      | varchar(10)  | YES  |     | ACTIVE   |                 |
| open_on     | date         | YES  |     | curdate()| DEFAULT_GENERATED|
+-------------+--------------+------+-----+----------+-----------------+
```

8 rows in set (0.00 sec)

```
mysql>
```

3. Problem statement
   Customer (CustomerID, Name, Address, Phone, Email)
   Account (AccountID, CustomerID, AccountType, Balance, CreatedDate)
   Transaction (TransactionID, AccountID, TransactionType, Amount, TransactionDate)
   1) Create a view to display customer name, account ID, and current balance.
   2) Create an index on the TransactionDate in the Transaction table.
   3) Constraints: Apply Primary Key on all IDs, Foreign Keys between Customer–Account and Account–Transaction.Use Check Constraint on Balance >= 0, Transaction Type in ('Credit', 'Debit'). Use Default Constraint on CreatedDate and TransactionDate as current date.

   ```
   CREATE DATABASE BankTransactionDB;
   Query OK, 1 row affected (0.00 sec)
   ```

```
mysql> USE BankTransactionDB;
Database changed
mysql> CREATE TABLE Customer (
    ->     CustomerID INT PRIMARY KEY,
    ->     Name VARCHAR(100) NOT NULL,
    ->     Address VARCHAR(200),
    ->     Phone VARCHAR(15) UNIQUE NOT NULL,
    ->     Email VARCHAR(120) UNIQUE
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Account (
    ->     AccountID INT PRIMARY KEY,
    ->     CustomerID INT NOT NULL,
    ->     AccountType VARCHAR(20) CHECK (AccountType IN
('Savings','Current','Fixed')),
    ->     Balance DECIMAL(12,2) CHECK (Balance >= 0),
    ->     CreatedDate DATE DEFAULT (CURRENT_DATE),
    ->     FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Transaction (
    ->     TransactionID INT PRIMARY KEY,
    ->     AccountID INT NOT NULL,
    ->     TransactionType VARCHAR(10) CHECK (TransactionType IN
('Credit','Debit')),
    ->     Amount DECIMAL(12,2) CHECK (Amount > 0),
    ->     TransactionDate DATE DEFAULT (CURRENT_DATE),
    ->     FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE VIEW CustomerAccountView AS
    -> SELECT
    ->     c.Name AS CustomerName,
    ->     a.AccountID,
    ->     a.Balance AS CurrentBalance
    -> FROM
    ->     Customer c
    -> JOIN
    ->     Account a ON c.CustomerID = a.CustomerID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM CustomerAccountView;
Empty set (0.00 sec)

mysql> CREATE INDEX idx_TransactionDate
    -> ON Transaction(TransactionDate);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
mysql> SHOW TABLES;
+---------------------------+
| Tables_in_banktransactiondb |
+---------------------------+
| account                   |
| customer                  |
| customeraccountview       |
| transaction               |
+---------------------------+
4 rows in set (0.00 sec)

mysql> DESC Customer;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| CustomerID | int          | NO   | PRI | NULL    |       |
| Name       | varchar(100) | NO   |     | NULL    |       |
| Address    | varchar(200) | YES  |     | NULL    |       |
| Phone      | varchar(15)  | NO   | UNI | NULL    |       |
| Email      | varchar(120) | YES  | UNI | NULL    |       |
+------------+--------------+------+-----+---------+-------+
5 rows in set (0.00 sec)

mysql> DESC Account;
+-------------+---------------+------+-----+-----------+-------------------+
| Field       | Type          | Null | Key | Default   | Extra             |
+-------------+---------------+------+-----+-----------+-------------------+
| AccountID   | int           | NO   | PRI | NULL      |                   |
| CustomerID  | int           | NO   | MUL | NULL      |                   |
| AccountType | varchar(20)   | YES  |     | NULL      |                   |
| Balance     | decimal(12,2) | YES  |     | NULL      |                   |
| CreatedDate | date          | YES  |     | curdate() | DEFAULT_GENERATED |
+-------------+---------------+------+-----+-----------+-------------------+
5 rows in set (0.00 sec)

mysql> DESC Transaction;
+-----------------+---------------+------+-----+-----------+-------------------+
| Field           | Type          | Null | Key | Default   | Extra             |
+-----------------+---------------+------+-----+-----------+-------------------+
| TransactionID   | int           | NO   | PRI | NULL      |                   |
| AccountID       | int           | NO   | MUL | NULL      |                   |
| TransactionType | varchar(10)   | YES  |     | NULL      |                   |
| Amount          | decimal(12,2) | YES  |     | NULL      |                   |
| TransactionDate | date          | YES  | MUL | curdate() | DEFAULT_GENERATED |
+-----------------+---------------+------+-----+-----------+-------------------+
5 rows in set (0.00 sec)

mysql> SHOW CREATE VIEW CustomerAccountView;
+--------------------+-------------------------
```

```
| View                | Create View
| character_set_client | collation_connection |
+---------------------+-------------------
```

3	2	1	2025-10-25	2025-11-08	*NULL*
4	3	4	2025-09-15	2025-09-29	2025-09-28
5	4	3	2025-10-01	2025-10-15	*NULL*

1. Display members and the books they have issued using INNER JOIN
2. Show all members and the books they issued using Left outer join
3. Show all books and who issued them using Right outer join.
4. Using Subquery (in WHERE clause) – Find members who issued books authored by 'Navathe'
5. Using Subquery (in SELECT clause) – Count of books issued by each Member

```
CREATE DATABASE LibraryDB;

Query OK, 1 row affected (0.01 sec)


mysql> USE LibraryDB;

Database changed

mysql> CREATE TABLE Books (

    ->     b_id INT PRIMARY KEY,

    ->     isbn VARCHAR(20) UNIQUE NOT NULL,

    ->     title VARCHAR(120) NOT NULL,

    ->     author VARCHAR(100) NOT NULL,

    ->     genre VARCHAR(50),

    ->     copies INT CHECK (copies >= 0)

    -> );

Query OK, 0 rows affected (0.03 sec)


mysql> CREATE TABLE Members (

    ->     m_id INT PRIMARY KEY,

    ->     m_name VARCHAR(100) NOT NULL,

    ->     email VARCHAR(100) UNIQUE NOT NULL,

    ->     phone VARCHAR(15) UNIQUE NOT NULL,
```

```
    ->     join_date DATE DEFAULT (CURRENT_DATE)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE IssuedBooks (
    ->     issue_id INT PRIMARY KEY,
    ->     m_id INT,
    ->     b_id INT,
    ->     issue_date DATE DEFAULT (CURRENT_DATE),
    ->     due_date DATE,
    ->     return_date DATE,
    ->     FOREIGN KEY (m_id) REFERENCES Members(m_id),
    ->     FOREIGN KEY (b_id) REFERENCES Books(b_id)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Books VALUES
    -> (1, '70777', 'Database System Concepts', 'Silberschatz', 'DBMS', 5),
    -> (2, '5398', 'Fundamentals of Database Systems', 'Elmasri & Navathe', 'DBMS', 4),
    -> (3, '3627', 'The C Programming Language', 'Kernighan & Ritchie', 'Programming', 3),
    -> (4, '7660', 'Designing Data-Intensive Applications', 'Kleppmann', 'Data', 2),
    -> (5, '30735', 'Let Us C', 'Yashavant Kanetkar', 'Programming', 6);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Members VALUES
    -> (1, 'Asha Patil', 'asha@library.test', '9876500011', '2025-06-10'),
```

```
    -> (2, 'Rahul Deshmukh', 'rahul@library.test', '9876500022', '2025-07-01'),
    -> (3, 'Priya Kulkarni', 'priya@library.test', '9876500033', '2025-07-15'),
    -> (4, 'Vikram Joshi', 'vikram@library.test', '9876500044', '2025-08-05');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO IssuedBooks VALUES
    -> (1, 1, 2, '2025-10-20', '2025-11-03', NULL),
    -> (2, 1, 5, '2025-09-10', '2025-09-24', '2025-09-20'),
    -> (3, 2, 1, '2025-10-25', '2025-11-08', NULL),
    -> (4, 3, 4, '2025-09-15', '2025-09-29', '2025-09-28'),
    -> (5, 4, 3, '2025-10-01', '2025-10-15', NULL);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT
    ->     Members.m_name AS MemberName,
    ->     Books.title AS BookTitle,
    ->     IssuedBooks.issue_date,
    ->     IssuedBooks.due_date
    -> FROM
    ->     Members
    -> INNER JOIN IssuedBooks ON Members.m_id = IssuedBooks.m_id
    -> INNER JOIN Books ON Books.b_id = IssuedBooks.b_id;
+---------------+-------------------------------------+------------+------------+
| MemberName    | BookTitle                           | issue_date | due_date   |
+---------------+-------------------------------------+------------+------------+
```

```
| Asha Patil     | Fundamentals of Database Systems      | 2025-10-20 | 2025-11-03 |
| Asha Patil     | Let Us C                              | 2025-09-10 | 2025-09-24 |
| Rahul Deshmukh | Database System Concepts              | 2025-10-25 | 2025-11-08 |
| Priya Kulkarni | Designing Data-Intensive Applications | 2025-09-15 | 2025-09-29 |
| Vikram Joshi   | The C Programming Language            | 2025-10-01 | 2025-10-15 |
+----------------+---------------------------------------+------------+------------+
5 rows in set (0.00 sec)

mysql> SELECT
    ->     Members.m_name AS MemberName,
    ->     Books.title AS BookTitle,
    ->     IssuedBooks.issue_date
    -> FROM
    ->     Members
    -> LEFT JOIN IssuedBooks ON Members.m_id = IssuedBooks.m_id
    -> LEFT JOIN Books ON Books.b_id = IssuedBooks.b_id;
+----------------+---------------------------------------+------------+
| MemberName     | BookTitle                             | issue_date |
+----------------+---------------------------------------+------------+
| Asha Patil     | Fundamentals of Database Systems      | 2025-10-20 |
| Asha Patil     | Let Us C                              | 2025-09-10 |
| Rahul Deshmukh | Database System Concepts              | 2025-10-25 |
| Priya Kulkarni | Designing Data-Intensive Applications | 2025-09-15 |
| Vikram Joshi   | The C Programming Language            | 2025-10-01 |
+----------------+---------------------------------------+------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT
    ->     Members.m_name AS MemberName,
    ->     Books.title AS BookTitle,
    ->     IssuedBooks.issue_date
    -> FROM
    ->     Members
    -> RIGHT JOIN IssuedBooks ON Members.m_id = IssuedBooks.m_id
    -> RIGHT JOIN Books ON Books.b_id = IssuedBooks.b_id;
+---------------+-------------------------------------+------------+
| MemberName    | BookTitle                           | issue_date |
+---------------+-------------------------------------+------------+
| Rahul Deshmukh | Database System Concepts           | 2025-10-25 |
| Asha Patil    | Fundamentals of Database Systems    | 2025-10-20 |
| Vikram Joshi  | The C Programming Language          | 2025-10-01 |
| Priya Kulkarni | Designing Data-Intensive Applications | 2025-09-15 |
| Asha Patil    | Let Us C                            | 2025-09-10 |
+---------------+-------------------------------------+------------+
5 rows in set (0.00 sec)


mysql> SELECT m_name
    -> FROM Members
    -> WHERE m_id IN (
    ->     SELECT m_id
    ->     FROM IssuedBooks
    ->     WHERE b_id IN (
    ->         SELECT b_id FROM Books WHERE author LIKE '%Navathe%'
    ->     )
```

```
    -> );
+------------+
| m_name     |
+------------+
| Asha Patil |
+------------+
1 row in set (0.01 sec)

mysql> SELECT
    ->     m_name,
    ->     (SELECT COUNT(*) FROM IssuedBooks WHERE Members.m_id =
IssuedBooks.m_id) AS BooksIssued
    -> FROM Members;
+---------------+-------------+
| m_name        | BooksIssued |
+---------------+-------------+
| Asha Patil    |           2 |
| Rahul Deshmukh |          1 |
| Priya Kulkarni |          1 |
| Vikram Joshi   |          1 |
+---------------+-------------+
4 rows in set (0.01 sec)

mysql> SHOW TABLES;
+--------------------+
| Tables_in_librarydb |
+--------------------+
```

```
| books              |
| issuedbooks        |
| members            |
+--------------------+
3 rows in set (0.01 sec)

mysql> DESC Books;
+--------+--------------+------+-----+---------+-------+
| Field  | Type         | Null | Key | Default | Extra |
+--------+--------------+------+-----+---------+-------+
| b_id   | int          | NO   | PRI | NULL    |       |
| isbn   | varchar(20)  | NO   | UNI | NULL    |       |
| title  | varchar(120) | NO   |     | NULL    |       |
| author | varchar(100) | NO   |     | NULL    |       |
| genre  | varchar(50)  | YES  |     | NULL    |       |
| copies | int          | YES  |     | NULL    |       |
+--------+--------------+------+-----+---------+-------+
6 rows in set (0.00 sec)

mysql> DESC Members;
+-----------+--------------+------+-----+-----------+-----------------+
| Field     | Type         | Null | Key | Default   | Extra           |
+-----------+--------------+------+-----+-----------+-----------------+
| m_id      | int          | NO   | PRI | NULL      |                 |
| m_name    | varchar(100) | NO   |     | NULL      |                 |
| email     | varchar(100) | NO   | UNI | NULL      |                 |
| phone     | varchar(15)  | NO   | UNI | NULL      |                 |
```

```
| join_date | date         | YES  |     | curdate() | DEFAULT_GENERATED |
+-----------+--------------+------+-----+-----------+-------------------+
5 rows in set (0.00 sec)

mysql> DESC IssuedBooks;
+-------------+------+------+-----+-----------+-------------------+
| Field       | Type | Null | Key | Default   | Extra             |
+-------------+------+------+-----+-----------+-------------------+
| issue_id    | int  | NO   | PRI | NULL      |                   |
| m_id        | int  | YES  | MUL | NULL      |                   |
| b_id        | int  | YES  | MUL | NULL      |                   |
| issue_date  | date | YES  |     | curdate() | DEFAULT_GENERATED |
| due_date    | date | YES  |     | NULL      |                   |
| return_date | date | YES  |     | NULL      |                   |
+-------------+------+------+-----+-----------+-------------------+
6 rows in set (0.00 sec)

mysql> SELECT * FROM Books;
+------+-------+-----------------------------------+--------------------+-------------+--------+
| b_id | isbn  | title                             | author             | genre       | copies |
+------+-------+-----------------------------------+--------------------+-------------+--------+
|    1 | 70777 | Database System Concepts          | Silberschatz       | DBMS        |      5 |
|    2 | 5398  | Fundamentals of Database Systems  | Elmasri & Navathe  | DBMS        |      4 |
|    3 | 3627  | The C Programming Language        | Kernighan & Ritchie | Programming |      3 |
|    4 | 7660  | Designing Data-Intensive Applications | Kleppmann      | Data        |      2 |
|    5 | 30735 | Let Us C                          | Yashavant Kanetkar | Programming |      6 |
```

```
+------+-------+-------------------------------------+-------------------+------------+--------+
```

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM Members;
+------+---------------+--------------------+-----------+-----------+
| m_id | m_name        | email              | phone     | join_date |
+------+---------------+--------------------+-----------+-----------+
|    1 | Asha Patil    | asha@library.test  | 9876500011 | 2025-06-10 |
|    2 | Rahul Deshmukh | rahul@library.test | 9876500022 | 2025-07-01 |
|    3 | Priya Kulkarni | priya@library.test | 9876500033 | 2025-07-15 |
|    4 | Vikram Joshi  | vikram@library.test | 9876500044 | 2025-08-05 |
+------+---------------+--------------------+-----------+-----------+
```

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM IssuedBooks;
+----------+------+------+------------+------------+-------------+
| issue_id | m_id | b_id | issue_date | due_date   | return_date |
+----------+------+------+------------+------------+-------------+
|        1 |    1 |    2 | 2025-10-20 | 2025-11-03 | NULL        |
|        2 |    1 |    5 | 2025-09-10 | 2025-09-24 | 2025-09-20  |
|        3 |    2 |    1 | 2025-10-25 | 2025-11-08 | NULL        |
|        4 |    3 |    4 | 2025-09-15 | 2025-09-29 | 2025-09-28  |
|        5 |    4 |    3 | 2025-10-01 | 2025-10-15 | NULL        |
+----------+------+------+------------+------------+-------------+
```

5 rows in set (0.00 sec)

```
mysql>
```

Problem statement 6:
Design and implement a Sales Management System database using SQL that maintains records of customers, products, and sales transactions.
The database contains the following tables:
Customer (customer_id, customer_name, city, email)
Product (product_id, product_name, category, price)
Sale (sale_id, customer_id, product_id, quantity, sale_date, total_amount)

A. Aggregation Operations

1) Calculate total sales amount per customer
2) Find total quantity sold for each product
3) Find the highest and lowest product prices
4)Find average sales amount per city
5) Find total sales and count of transactions per category

B. Subquery Operations

1) Display details of customers whose total purchase amount is greater than the average sales amount of all customers.
2) Create a temporary summary table showing each customer's total sales, and display only those who purchased more than ₹10,000.
3) Display each product along with the number of times it was sold.
4) List customers who have purchased at least one product in the 'Electronics' category.

```
mysql> CREATE DATABASE ShopDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE ShopDB;
Database changed
mysql> CREATE TABLE Customer(
    ->   customer_id INT PRIMARY KEY,
    ->   customer_name VARCHAR(50),
    ->   city VARCHAR(50),
    ->   email VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Product(
    ->   product_id INT PRIMARY KEY,
    ->   product_name VARCHAR(50),
    ->   category VARCHAR(50),
    ->   price DECIMAL(10,2)
    -> );
```

```
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE TABLE Sale(
    ->   sale_id INT PRIMARY KEY,
    ->   customer_id INT,
    ->   product_id INT,
    ->   quantity INT,
    ->   sale_date DATE,
    ->   total_amount DECIMAL(10,2),
    ->   FOREIGN KEY(customer_id) REFERENCES Customer(customer_id),
    ->   FOREIGN KEY(product_id) REFERENCES Product(product_id)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO Customer VALUES
    -> (1,'Asha','Pune','asha@shop.test'),
    -> (2,'Rahul','Mumbai','rahul@shop.test'),
    -> (3,'Priya','Nashik','priya@shop.test'),
    -> (4,'Vikram','Pune','vikram@shop.test');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Product VALUES
    -> (101,'Laptop','Electronics',55000),
    -> (102,'Mobile','Electronics',25000),
    -> (103,'Table','Furniture',5000),
    -> (104,'Chair','Furniture',2000),
    -> (105,'Washing Machine','Appliances',18000);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Sale VALUES
    -> (1,1,101,1,'2025-10-01',55000),
    -> (2,2,102,2,'2025-10-03',50000),
    -> (3,3,103,1,'2025-10-05',5000),
    -> (4,1,104,2,'2025-10-07',4000),
    -> (5,4,105,1,'2025-10-08',18000),
    -> (6,2,103,3,'2025-10-09',15000);
Query OK, 6 rows affected (0.00 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT customer_id, SUM(total_amount) AS total_sales
    -> FROM Sale
```

```
    -> GROUP BY customer_id;
+-------------+-------------+
| customer_id | total_sales |
+-------------+-------------+
|           1 |    59000.00 |
|           2 |    65000.00 |
|           3 |     5000.00 |
|           4 |    18000.00 |
+-------------+-------------+
4 rows in set (0.00 sec)

mysql> SELECT product_id, SUM(quantity) AS total_qty
    -> FROM Sale
    -> GROUP BY product_id;
+------------+-----------+
| product_id | total_qty |
+------------+-----------+
|        101 |         1 |
|        102 |         2 |
|        103 |         4 |
|        104 |         2 |
|        105 |         1 |
+------------+-----------+
5 rows in set (0.00 sec)

mysql> SELECT MAX(price) AS highest, MIN(price) AS lowest FROM Product;
+----------+---------+
| highest  | lowest  |
+----------+---------+
| 55000.00 | 2000.00 |
+----------+---------+
1 row in set (0.00 sec)

mysql> SELECT c.city, AVG(s.total_amount) AS avg_sales
    -> FROM Sale s, Customer c
    -> WHERE s.customer_id = c.customer_id
    -> GROUP BY c.city;
+--------+-------------+
| city   | avg_sales   |
+--------+-------------+
| Pune   | 25666.666667 |
| Mumbai | 32500.000000 |
| Nashik |  5000.000000 |
+--------+-------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT p.category, SUM(s.total_amount) AS total_sales, COUNT(s.sale_id)
AS transactions
    -> FROM Sale s, Product p
    -> WHERE s.product_id = p.product_id
    -> GROUP BY p.category;
+-------------+-------------+--------------+
| category    | total_sales | transactions |
+-------------+-------------+--------------+
| Electronics |   105000.00 |            2 |
| Furniture   |    24000.00 |            3 |
| Appliances  |    18000.00 |            1 |
+-------------+-------------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT customer_id
    -> FROM Sale
    -> GROUP BY customer_id
    -> HAVING SUM(total_amount) > (SELECT AVG(total_amount) FROM Sale);
+-------------+
| customer_id |
+-------------+
|           1 |
|           2 |
+-------------+
2 rows in set (0.00 sec)

mysql> SELECT customer_id, SUM(total_amount) AS total_sales
    -> FROM Sale
    -> GROUP BY customer_id
    -> HAVING total_sales > 10000;
+-------------+-------------+
| customer_id | total_sales |
+-------------+-------------+
|           1 |    59000.00 |
|           2 |    65000.00 |
|           4 |    18000.00 |
+-------------+-------------+
3 rows in set (0.00 sec)

mysql> SELECT product_id, COUNT(sale_id) AS times_sold
    -> FROM Sale
    -> GROUP BY product_id;
+------------+------------+
| product_id | times_sold |
+------------+------------+
|        101 |          1 |
```

```
|        102 |          1 |
|        103 |          2 |
|        104 |          1 |
|        105 |          1 |
+------------+------------+
5 rows in set (0.00 sec)

mysql> SELECT DISTINCT customer_id
    -> FROM Sale
    -> WHERE product_id IN (SELECT product_id FROM Product WHERE
category='Electronics');
+-------------+
| customer_id |
+-------------+
|           1 |
|           2 |
+-------------+
2 rows in set (0.00 sec)

mysql>
```

Problem statement 7:

Unnamed PL/SQL code block: Use of Control structure and Exception handling is mandatory.
Suggested Problem statement:
Consider Tables:1. Borrower (Roll_no, Name, DateofIssue, NameofBook, Status)
2. Fine(Roll_no,Date,Amt)
Accept Roll_no and NameofBook from user.

Check the number of days (from date of issue). If days are between 15 to 30 then fine amount will be Rs 5per day. If no. of days>30, per day fine will be Rs 50 per day and for days less than 30, Rs. 5 perday. After submitting the book, status will change from I to R. If condition of fine is true, then details will be stored into fine table. Also handles the exception by named exception handler or user define exception handler

```
CREATE TABLE BookIssue (
  RollNo NUMBER PRIMARY KEY,
  StudentName VARCHAR2(50),
  IssueDate DATE,
  BookName VARCHAR2(50),
  Status CHAR(1)
);
```

```
CREATE TABLE Penalty (
  RollNo NUMBER,
  PenaltyDate DATE,
  Amount NUMBER
);

SET SERVEROUTPUT ON;
DECLARE
  rno NUMBER;              -- Roll number
  bname VARCHAR2(50);      -- Book name
  issuedate DATE;          -- Date of issue
  days NUMBER;             -- Days since issue
  fine NUMBER := 0;        -- Fine amount

BEGIN
  -- Take user input
  rno := &roll_no;
  bname := '&book_name';

  -- Get issue date from BookIssue table
  SELECT IssueDate INTO issuedate
  FROM BookIssue
  WHERE RollNo = rno AND BookName = bname AND Status = 'I';

  -- Calculate number of days book kept
  days := TRUNC(SYSDATE - issuedate);

  -- Fine calculation logic
  IF days > 30 THEN
     fine := days * 50;
  ELSIF days >= 15 THEN
     fine := days * 5;
  ELSE
     fine := 0;
  END IF;
```

```
  -- Update status to Returned
  UPDATE BookIssue
  SET Status = 'R'
  WHERE RollNo = rno AND BookName = bname;

  -- Insert fine record if applicable
  IF fine > 0 THEN
    INSERT INTO Penalty (RollNo, PenaltyDate, Amount)
    VALUES (rno, SYSDATE, fine);
    DBMS_OUTPUT.PUT_LINE('Fine applied: Rs ' || fine);
  ELSE
    DBMS_OUTPUT.PUT_LINE('No fine. Returned on time.');
  END IF;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('❌ Record not found for given RollNo or
BookName.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('⚠□ Error: ' || SQLERRM);
END;
/
```

Problem statement 8:
Write a Stored Procedure namely proc_Grade for the categorization of student. If marks scored bystudents in examination is <=1500 and marks>=990 then student will be placed in distinctioncategory if marks scored are between 989 and 900 category is first class, if marks 899 and 825 category is Higher Second Class. Write a PL/SQL block to use procedure created with above requirement. Stud_Marks(name, total_marks) Result(Roll,Name, Class)

```
SET SERVEROUTPUT ON;

-- Step 1: Create tables
CREATE TABLE Stud_Marks (
  name VARCHAR2(50),
```

```
  total_marks NUMBER
);

CREATE TABLE Result (
  roll NUMBER,
  name VARCHAR2(50),
  class VARCHAR2(30)
);

-- Step 2: Insert sample data
INSERT INTO Stud_Marks VALUES ('Asha', 1200);
INSERT INTO Stud_Marks VALUES ('Ravi', 950);
INSERT INTO Stud_Marks VALUES ('Neha', 870);
INSERT INTO Stud_Marks VALUES ('Karan', 800);
COMMIT;

-- Step 3: Create simple procedure
CREATE OR REPLACE PROCEDURE proc_Grade(p_name VARCHAR2, p_marks
NUMBER) AS
  grade VARCHAR2(30);
BEGIN
  IF p_marks >= 990 AND p_marks <= 1500 THEN
     grade := 'Distinction';
  ELSIF p_marks BETWEEN 900 AND 989 THEN
     grade := 'First Class';
  ELSIF p_marks BETWEEN 825 AND 899 THEN
     grade := 'Higher Second Class';
  ELSE
     grade := 'Fail';
  END IF;

  INSERT INTO Result VALUES (1, p_name, grade);
  DBMS_OUTPUT.PUT_LINE(p_name || ' got ' || grade);
END;
/
```

```
-- Step 4: Run procedure for one student (you can change name & marks)
BEGIN
  proc_Grade('Asha', 1200);
  proc_Grade('Ravi', 950);
  proc_Grade('Neha', 870);
  proc_Grade('Karan', 800);
END;
/

-- Step 5: See final result
SELECT * FROM Result;
```

Problem statement 9:

Cursors: (All types: Implicit, Explicit, Cursor FOR Loop, Parameterized Cursor)

Problem Statement: Student_Result using Cursors

Student_Result(stud_id, name,dept,Total_mark)

1) Write a PL/SQL block to insert a new record into Student_Result the table and display the number of rows affected
2) display the name and Total_marks of each student from the Student_Result table.
3) Use FOR loop to display the data
4) Create a **parameterized cursor** that accepts the *department name* as a parameter and displays the list of students
5) Write the cursor for UPDATE

```
-- Create table (Run only once)
CREATE TABLE Student_Result (
 sid NUMBER,
 sname VARCHAR2(20),
 dept VARCHAR2(20),
 total NUMBER
);

-- PL/SQL block
DECLARE
 v_count NUMBER;      -- to store affected rows
 v_name  VARCHAR2(20);
 v_mark  NUMBER;

 -- Explicit cursor
 CURSOR c1 IS SELECT sname, total FROM Student_Result;

 -- Parameterized cursor
 CURSOR c_dept(p_dept VARCHAR2) IS
   SELECT sname, total FROM Student_Result WHERE dept = p_dept;

 -- Cursor for update
```

```
 CURSOR c_up IS SELECT total FROM Student_Result FOR UPDATE;
BEGIN
 -- 1) Insert records
 INSERT INTO Student_Result VALUES (1, 'Amit', 'Computer', 1200);
 INSERT INTO Student_Result VALUES (2, 'Riya', 'IT', 980);
 INSERT INTO Student_Result VALUES (3, 'Rohan', 'Computer', 1110);
 v_count := SQL%ROWCOUNT;
 DBMS_OUTPUT.PUT_LINE(v_count || ' rows inserted.');

 -- 2) Display name and marks using explicit cursor
 DBMS_OUTPUT.PUT_LINE(CHR(10) || '--- Using Explicit Cursor ---');
 OPEN c1;
 LOOP
  FETCH c1 INTO v_name, v_mark;
  EXIT WHEN c1%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || '  Marks: ' || v_mark);
 END LOOP;
 CLOSE c1;

 -- 3) Using FOR loop
 DBMS_OUTPUT.PUT_LINE(CHR(10) || '--- Using FOR Loop ---');
 FOR rec IN (SELECT sname, total FROM Student_Result) LOOP
  DBMS_OUTPUT.PUT_LINE('Name: ' || rec.sname || '  Marks: ' || rec.total);
 END LOOP;

 -- 4) Parameterized cursor
 DBMS_OUTPUT.PUT_LINE(CHR(10) || '--- Computer Dept Students ---');
 FOR rec IN c_dept('Computer') LOOP
  DBMS_OUTPUT.PUT_LINE('Name: ' || rec.sname || '  Marks: ' || rec.total);
 END LOOP;

 -- 5) Cursor for update
 DBMS_OUTPUT.PUT_LINE(CHR(10) || '--- Updating Marks (Add 10) ---');
 FOR rec IN c_up LOOP
  v_mark := rec.total + 10;
  UPDATE Student_Result
  SET total = v_mark
  WHERE CURRENT OF c_up;
 END LOOP;

 DBMS_OUTPUT.PUT_LINE('Marks updated successfully.');
END;
/
```

Problem statement 10:

Create a Library database with the schema Books(AccNo,Title,Author,Publisher,Count).

a. Create a table Library_Audit with same fields as of Books.
b. Create a before trigger to insert records into Librry_Audit table if there is deletion in Books table.

   Create a after trigger to insert records into Librry_Audit table if there is updation in Books table.

```
-- Create main table
```

```
CREATE TABLE Books(
 accno NUMBER,
 title VARCHAR2(30),
 author VARCHAR2(30),
 publisher VARCHAR2(30),
 count NUMBER
);

-- Create audit table
CREATE TABLE Library_Audit(
 accno NUMBER,
 title VARCHAR2(30),
 author VARCHAR2(30),
 publisher VARCHAR2(30),
 count NUMBER,
 action VARCHAR2(20)
);

-- Before Delete Trigger
CREATE OR REPLACE TRIGGER trg_before_del
BEFORE DELETE ON Books
FOR EACH ROW
BEGIN
 INSERT INTO Library_Audit
 VALUES(:OLD.accno, :OLD.title, :OLD.author, :OLD.publisher, :OLD.count,
'Deleted');
END;
/

-- After Update Trigger
CREATE OR REPLACE TRIGGER trg_after_upd
AFTER UPDATE ON Books
FOR EACH ROW
BEGIN
 INSERT INTO Library_Audit
 VALUES(:OLD.accno, :OLD.title, :OLD.author, :OLD.publisher, :OLD.count,
'Updated');
END;
/

-- Test data
INSERT INTO Books VALUES(1,'DBMS','Korth','McGraw',5);
INSERT INTO Books VALUES(2,'C Prog','Dennis','Pearson',3);
```

```
-- Perform update and delete to test triggers
UPDATE Books SET count=6 WHERE accno=1;
DELETE FROM Books WHERE accno=2;

-- Check audit table
SELECT * FROM Library_Audit;
```

Problem statement 11:
Write a program to implement mysql database connectivity with java to implement Database navigation operations (add, delete, edit, fetch)

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Books (
  id INT PRIMARY KEY,
  title VARCHAR(50),
  author VARCHAR(50)
);
```

```
import java.sql.*;
import java.util.Scanner;

public class LibraryDBDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Connection con = null;
        Statement st = null;
        ResultSet rs = null;

        try {
            // 1. Load JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // 2. Connect to MySQL database
            con = DriverManager.getConnection("jdbc:mysql://localhost:3306/CollegeDB",
"root", "root");
            st = con.createStatement();

            int choice;
            do {
                System.out.println("\n--- Library Menu ---");
```

```
            System.out.println("1. Add Book");
            System.out.println("2. Edit Book");
            System.out.println("3. Delete Book");
            System.out.println("4. View All Books");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine(); // clear buffer

            switch (choice) {
                case 1:
                    System.out.print("Enter ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();
                    System.out.print("Enter Title: ");
                    String title = sc.nextLine();
                    System.out.print("Enter Author: ");
                    String author = sc.nextLine();

                    st.executeUpdate("INSERT INTO Books VALUES(" + id + ", '" + title + "', '"
+ author + "')");
                    System.out.println("Book added successfully!");
                    break;

                case 2:
                    System.out.print("Enter Book ID to update: ");
                    int uid = sc.nextInt();
                    sc.nextLine();
                    System.out.print("Enter New Title: ");
                    String newTitle = sc.nextLine();
                    System.out.print("Enter New Author: ");
                    String newAuthor = sc.nextLine();

                    st.executeUpdate("UPDATE Books SET title='" + newTitle + "', author='" +
newAuthor + "' WHERE id=" + uid);
                    System.out.println("Book updated successfully!");
                    break;

                case 3:
                    System.out.print("Enter Book ID to delete: ");
                    int did = sc.nextInt();
                    st.executeUpdate("DELETE FROM Books WHERE id=" + did);
```

```
                System.out.println("Book deleted successfully!");
                break;

            case 4:
                rs = st.executeQuery("SELECT * FROM Books");
                System.out.println("\n--- Book List ---");
                while (rs.next()) {
                    System.out.println("ID: " + rs.getInt("id") +
                                       ", Title: " + rs.getString("title") +
                                       ", Author: " + rs.getString("author"));
                }
                break;

            case 5:
                System.out.println("Exiting...");
                break;

            default:
                System.out.println("Invalid choice!");
            }
        } while (choice != 5);

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try {
            if (con != null) con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Problem statement 12:

MongoDB Queries: Design and Develop MongoDB Queries using CRUD operations. (Use CRUD operations, SAVE method, logical operators etc.).

Implement the following MongoDb Query

1. Create a collection named books.
2. Insert 5 records with field TITLE,DESCRIPTION,BY,URL,TAGS AND LIKES
3. Insert 1 more document in collection with additional field of user name and comments.
4. Display all the documents whose title is 'mongodb'.

5. Display all the documents written by 'john' or whose title is 'mongodb'.
6. Display all the documents whose title is 'mongodb' and written by 'john'.
7. Display all the documents whose like is greater than 10.
8. Display all the documents whose like is greater than 100 and whose title is either 'mongodb' or written by 'john'.
9. Update the title of 'mongodb' document to 'mongodb overview'
10. Delete the document titled 'nosql overview'.
11. Display exactly two documents written by 'john'.
12. Display the second document published by 'john'.
13. Display all the books in the sorted fashion.

```
//create database
db.createCollection(“books”)

// 1. Create / Switch to database
use LibraryDB

// 2. Create a collection named books and insert 5 records
db.books.insertMany([
  {
    title: "mongodb",
    description: "Database basics",
    by: "john",
    url: "https://mongodb.com",
    tags: ["database", "nosql"],
    likes: 50
  },
  {
    title: "nodejs",
    description: "Server-side JS",
    by: "peter",
    url: "https://nodejs.org",
    tags: ["javascript", "backend"],
    likes: 20
  },
  {
    title: "nosql overview",
    description: "About NoSQL",
    by: "mike",
    url: "https://nosql.org",
    tags: ["database", "overview"],
    likes: 8
  },
  {
    title: "expressjs",
    description: "Web framework",
    by: "john",
    url: "https://expressjs.com",
    tags: ["nodejs", "web"],
    likes: 15
  },
  {
    title: "reactjs",
    description: "Frontend library",
```

```
    by: "anna",
    url: "https://react.dev",
    tags: ["frontend", "ui"],
    likes: 120
  }
]);

// 3. Insert one more document with extra fields
db.books.insertOne({
  title: "mongodb advanced",
  description: "Advanced topics in MongoDB",
  by: "john",
  url: "https://mongodb.com/advanced",
  tags: ["database", "advanced"],
  likes: 90,
  username: "admin",
  comments: "Very useful tutorial"
});

// 4. Display all documents whose title is 'mongodb'
db.books.find({ title: "mongodb" });

// 5. Display all documents written by 'john' OR whose title is 'mongodb'
db.books.find({ $or: [{ by: "john" }, { title: "mongodb" }] });

// 6. Display all documents whose title is 'mongodb' AND written by 'john'
db.books.find({ title: "mongodb", by: "john" });

// 7. Display all documents whose likes > 10
db.books.find({ likes: { $gt: 10 } });

// 8. Display all documents whose likes > 100 AND (title = 'mongodb' OR by =
'john')
db.books.find({
  likes: { $gt: 100 },
  $or: [{ title: "mongodb" }, { by: "john" }]
});

// 9. Update title 'mongodb' to 'mongodb overview'
db.books.updateOne(
  { title: "mongodb" },
  { $set: { title: "mongodb overview" } }
);

// 10. Delete document titled 'nosql overview'
db.books.deleteOne({ title: "nosql overview" });

// 11. Display exactly two documents written by 'john'
db.books.find({ by: "john" }).limit(2);

// 12. Display the second document published by 'john'
db.books.find({ by: "john" }).skip(1).limit(1);
```

```
// 13. Display all books in sorted fashion (by title ascending)
db.books.find().sort({ title: 1 });
```

Problem statement 13:

MongoDB - Aggregation and Indexing: Design and Develop MongoDB Queries using aggregation and indexing with suitable example using MongoDB.

Create the collection Books having the following fields TITLE, DESCRIPTION, BY, URL, TAGS AND LIKES.

Implement the following Aggregation and Indexing Queries

1. Find the number of books published by john.
2. Find books which have minimum likes and maximum likes published by john.
3. Find the average number of likes of the books published by john.
4. Find the first and last book published by john.
5. Create an index on author name.

Display the books published by john and check if it uses the index which we have created

```
// 1 Use / Create Database
use LibraryDB

// 2 Create collection and insert sample data
db.books.insertMany([
  { title: "mongodb basics", description: "Intro to MongoDB", by: "john", url: "https://mongodb.com", tags: ["db"], likes: 20 },
  { title: "mongodb advanced", description: "Advanced MongoDB", by: "john", url: "https://mongodb.com/adv", tags: ["db"], likes: 80 },
  { title: "nodejs", description: "Server side JS", by: "peter", url: "https://nodejs.org", tags: ["backend"], likes: 40 },
  { title: "expressjs", description: "Web framework", by: "john", url: "https://expressjs.com", tags: ["web"], likes: 10 },
  { title: "reactjs", description: "Frontend library", by: "anna", url: "https://react.dev", tags: ["frontend"], likes: 100 }
]);

// ------------------------------------------------
// 🔷 Aggregation Queries
// ------------------------------------------------

// 1 Find the number of books published by john
db.books.aggregate([
  { $match: { by: "john" } },
  { $count: "Total_Books_By_John" }
]);
```

```
// 2️⃣ Find books with MIN and MAX likes published by john
db.books.aggregate([
  { $match: { by: "john" } },
  {
    $group: {
      _id: "$by",
      minLikes: { $min: "$likes" },
      maxLikes: { $max: "$likes" }
    }
  }
]);

// 3️⃣ Find average number of likes of books published by john
db.books.aggregate([
  { $match: { by: "john" } },
  {
    $group: {
      _id: "$by",
      avgLikes: { $avg: "$likes" }
    }
  }
]);

// 4️⃣ Find first and last book published by john (sorted by title)
db.books.aggregate([
  { $match: { by: "john" } },
  { $sort: { title: 1 } },
  {
    $group: {
      _id: "$by",
      firstBook: { $first: "$title" },
      lastBook: { $last: "$title" }
    }
  }
]);

// ---------------------------------------------
// 🔹 Indexing
// ---------------------------------------------

// 5️⃣ Create an index on author name
db.books.createIndex({ by: 1 });
```

```
// ✅ Check all indexes
db.books.getIndexes();

// ✅ Display books published by john (index will be used)
db.books.find({ by: "john" }).explain("executionStats");
```

Problem statement 14:

MongoDB - Map reduces operations: Implement Map reduces operation with suitable example using MongoDB Create the following schema

Order (id, amount ,status)

Order={{cus_id:A1,amount:400,status:P},{cus_id:B1,amount:300,status:D},{cus_id:A1,amount:200,status:F},{cus_id:C1,amount:200,status:F},{cus_id:B1,amount:700,status:P},{cus_id:B1,amount:800,status:P}}

Status: P=”Pending”, D= “Delivered”, F= “Failed”

Implement the following using Map Reduce function

1. Find the sum of amount of each customer whose status is P
2. Find the average amount of each customer
3. Find the min amount of each customer
4. Find the max amount of each customer whose status is F

```
// 1️⃣ Use / Create Database
use SalesDB

// 2️⃣ Create collection and insert sample data
db.orders.insertMany([
  { cus_id: "A1", amount: 400, status: "P" },
  { cus_id: "B1", amount: 300, status: "D" },
  { cus_id: "A1", amount: 200, status: "F" },
  { cus_id: "C1", amount: 200, status: "F" },
  { cus_id: "B1", amount: 700, status: "P" },
  { cus_id: "B1", amount: 800, status: "P" }
]);

// ------------------------------------------------------
// 1️⃣ Find sum of amount of each customer whose status is P
// ------------------------------------------------------
var mapSum = function() {
  if (this.status === "P") {
    emit(this.cus_id, this.amount);
  }
};

var reduceSum = function(key, values) {
  return Array.sum(values);
};

db.orders.mapReduce(mapSum, reduceSum, { out: "SumPending" });
```

```
print("Sum of amounts (status = P):");
printjson(db.SumPending.find().toArray());

// ---------------------------------------------------
// 2 Find average amount of each customer
// ---------------------------------------------------
var mapAvg = function() {
  emit(this.cus_id, { total: this.amount, count: 1 });
};

var reduceAvg = function(key, values) {
  var result = { total: 0, count: 0 };
  values.forEach(v => {
    result.total += v.total;
    result.count += v.count;
  });
  return result;
};

var finalizeAvg = function(key, reducedVal) {
  reducedVal.avg = reducedVal.total / reducedVal.count;
  return reducedVal;
};

db.orders.mapReduce(mapAvg, reduceAvg, { out: "AvgAmount", finalize:
finalizeAvg });
print("Average amount of each customer:");
printjson(db.AvgAmount.find().toArray());

// ---------------------------------------------------
// 3 Find minimum amount of each customer
// ---------------------------------------------------
var mapMin = function() {
  emit(this.cus_id, this.amount);
};

var reduceMin = function(key, values) {
  return Math.min.apply(null, values);
};

db.orders.mapReduce(mapMin, reduceMin, { out: "MinAmount" });
print("Minimum amount of each customer:");
printjson(db.MinAmount.find().toArray());

// ---------------------------------------------------
// 4 Find maximum amount of each customer whose status is F
// ---------------------------------------------------
var mapMax = function() {
  if (this.status === "F") {
    emit(this.cus_id, this.amount);
  }
};
```

```
var reduceMax = function(key, values) {
  return Math.max.apply(null, values);
};

db.orders.mapReduce(mapMax, reduceMax, { out: "MaxFailed" });
print("Maximum amount of each customer (status = F):");
printjson(db.MaxFailed.find().toArray());
```